

Class Inheritance

CSCI 1101

2024-04-23

Classes

Recall that a **Class** is a new mutable type that we can define.

An **interface** for a class is a collection of **methods** that allow us to interact with its **instance** objects.

Among these is an **initializer** that lets us set the state of newly created objects.

Typically, there are also **get** and **set** methods to query and update an object's state.

```
class MyClass :  
    def __init__(self , <more_args>) :  
        # initialize the state of a new object  
    def get_property(self) :  
        # get some information about the state of an object  
    def set_property(self , property) :  
        # set some information about the state of an object  
    etc.
```

Inheritance

It is common to want to create a number of classes that share certain similarities.

Python provides a mechanism to factor out these commonalities into one class and have other classes **derive** or **inherit** from it, each making the desired customizations.

Such a **derived class** is called a “subclass” of its **base class**.

Conceptually, the situation looks like this:

base class

derived class 1

derived class 2

...

A Family of Shapes

Suppose that we want to build some shapes that turtle can use for drawing.

We could organize these as follows:

Shape

Circle

Square

Triangle

...

Each shape will have attributes including its position, size, and color, and will have a method to draw itself.

The Base Class

The base class for our family contains the functionality common to all shapes:

```
class Shape :
```

```
    def __init__(self , position , size , color) :  
        self.position = position  
        self.size = size  
        self.color = color
```

```
    def draw(self) :  
        turtle.penup()  
        turtle.goto(self.position)  
        turtle.pencolor(self.color)  
        turtle.pendown()  
        # nothing to draw yet
```

Derived Classes

In order to indicate that one class is derived from another, we specify the name of the *base class* as a parameter.

By default, a derived class **inherits** the methods of its base class.

However, a derived class may instead **override** a base class method in order to provide different functionality.

A derived class may also define new methods, not present in its base class.

Derived classes can call base class methods using the `super` function.

```
class Circle(Shape) :  
    # inherits __init__  
  
    def draw(self) :    # Circle.draw overrides Shape.draw  
        super().draw() # calls Shape.draw(self)  
        turtle.circle(self.size)
```

Activity: Square (part 1)

Define a class `Square` that is derived from `Shape`.

Override the `draw` method to draw a square.

Hint: your `draw` method should call the base class `draw` method to get the turtle in the right place and ready to draw.

Class Hierarchies

Originally, we had a hierarchy of shapes that looks like this:

- Shape

 - Circle

 - Square

 - Triangle

But an equilateral triangle and a square have a lot in common: they are both **regular polygons**. So we can adjust our class hierarchy to look like this:

- Shape

 - Circle

 - Polygon

 - Triangle

 - Square

Polygons

Here is a class for polygons that derives from Shape.

```
class Polygon(Shape) :  
  
    # override the initializer with an additional argument:  
    def __init__(self , sides , position , size , color) :  
        self.sides = sides  
        super().__init__(position , size , color)  
  
    # override the draw method to draw a polygon:  
    def draw(self) :  
        super().draw()  
        for _ in range(self.sides) :  
            turtle.forward(self.size)  
            turtle.left(360/self.sides)
```

Triangles

We can make a class for triangles that derives from `Polygon` instead of directly from `Shape`.

```
class Triangle(Polygon) :  
  
    def __init__(self , position , size , color) :  
        super().__init__(3 , position , size , color)
```

- We override the initializer to omit the number of sides:
a triangle always has 3 sides.
- We don't need to override the `draw` method at all:
drawing an equilateral triangle *is* just drawing a regular polygon with 3 sides.

Activity: Square (part 2)

Redefine your `Square` class so that it is derived from `Polygon` rather than `Shape`.

Do the minimum work possible.

Adding Base Functionality

Methods and attributes added to a base class are available in its derived classes.

```
class Shape :
    def __init__(self , position , size , color) :
        self.position = position
        self.size = size
        self.color = color
        self.angle = 0                # new
    def set_angle(self , angle) :      # new
        self.angle = angle
    def draw(self) :
        turtle.penup()
        turtle.goto(self.position)
        turtle.setheading(self.angle)  # new
        turtle.pencolor(self.color)
        turtle.pendown()
```

Activity: Line Width

Modify your `Shape` class by adding a method `Shape.set_line_width` that takes an integer and stores it in an attribute.

Then modify the `Shape.draw` method so that it sets turtle's `pensize` to the value of this attribute.

If you do this right then you should be able to draw any `Shape` instance using the line width of your choice.

To Do

Finish *Homework 11: Recursion* on CodeRunner by this 11:00 this Thursday.